PYTHON UNIT 2

Python String

- A string is a sequence of characters. Python treats anything inside quotes as a string. This includes letters, numbers, and symbols. Python has no character data type so single character is a string of length 1.

```
s = "GfG"

print(s[1]) # access 2nd char
s1 = s + s[0] # update
print(s1) # print
```

Output

```
f

GfGG
```

Creating a String

Strings can be created using either **single (‘)** or **double (“)** quotes.

```
s1 = 'GfG'
s2 = "GfG"
print(s1)
print(s2)
```

Output

```
GfG

GfG
```

Multi-line Strings

If we need a string to span multiple lines then we can use **triple quotes (”’ or “””)**.

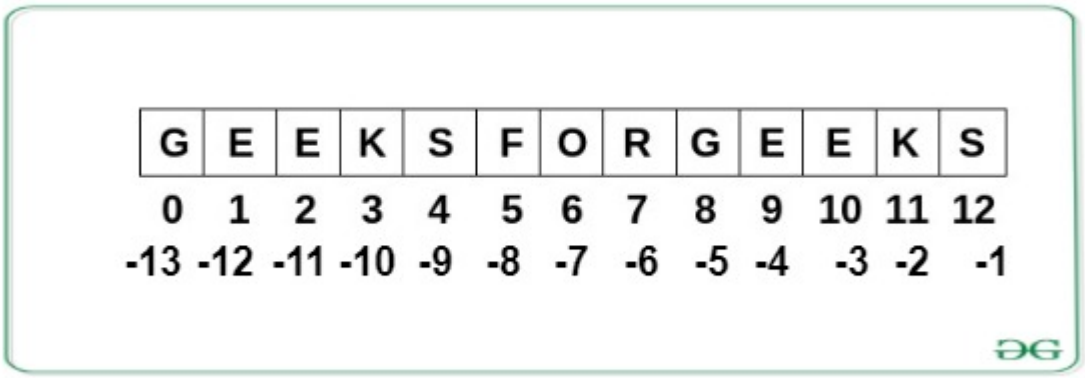
```
s = """I am Learning
Python String on GeeksforGeeks"""
print(s)

s = '''I'm a
Geek'''
print(s)
```

Output

```
I am Learning

Python String on GeeksforGeeks

I'm a

Geek
```

Accessing characters in Python String

Strings in Python are sequences of characters, so we can access individual characters using **indexing.** Strings are indexed starting from **0** and **-1** from end**.** This allows us to retrieve specific characters from the string.



Python String syntax indexing

```
s = "GeeksforGeeks"

# Accesses first character: 'G'
print(s[0])

# Accesses 5th character: 's'
print(s[4])
```

Output

```
G

s
```

Note: Accessing an index out of range will cause an **IndexError**. Only integers are allowed as indices and using a float or other types will result in a **TypeError**.

Access string with Negative Indexing

Python allows negative address references to access characters from back of the String, e.g. -1 refers to the last character, -2 refers to the second last character, and so on.

```
s = "GeeksforGeeks"

# Accesses 3rd character: 'k'
print(s[-10])

# Accesses 5th character from end: 'G'
print(s[-5])
```

Output

```
k

G
```

String Slicing

Slicing is a way to extract portion of a string by specifying the **start** and **end** indexes. The syntax for slicing is **string[start:end]**, where **start** starting index and **end** is stopping index (excluded).

```
s = "GeeksforGeeks"

# Retrieves characters from index 1 to 3: 'eek'
print(s[1:4])

# Retrieves characters from beginning to index 2: 'Gee'
print(s[:3])

# Retrieves characters from index 3 to the end: 'ksforGeeks'
print(s[3:])

# Reverse a string
print(s[::-1])
```

Output

```
eek

Gee

ksforGeeks

skeeGrofskeeG
```

String Immutability

Strings in Python are immutable. This means that they cannot be changed after they are created. If we need to manipulate strings then we can use methods like **concatenation, slicing,** or **formatting** to create new strings based on the original.

```
s = "geeksforGeeks"

# Trying to change the first character raises an error
# s[0] = 'I'  # Uncommenting this line will cause a TypeError

# Instead, create a new string
s = "G" + s[1:]
print(s)
```

Output

```
GeeksforGeeks
```

Deleting a String

In Python, it is not possible to delete individual characters from a string since strings are immutable. However, we can delete an entire string variable using the **del** keyword.

```
s = "GfG"

# Deletes entire string
del s
```

Note: After deleting the string using **del** and if we try to access **s** then it will result in a **NameError** because the variable no longer exists.

Updating a String

To update a part of a string we need to create a new string since strings are immutable.

```
s = "hello geeks"

# Updating by creating a new string
s1 = "H" + s[1:]

# replacnig "geeks" with "GeeksforGeeks"
s2 = s.replace("geeks", "GeeksforGeeks")
print(s1)
print(s2)
```

Output

```
Hello geeks

hello GeeksforGeeks
```

Explanation:

- **For s1,** The original string **s** is sliced from index 1 to end of string and then concatenate "H" to create a new string **s1**.
- **For s2,** we can created a new string s2 and used replace() method to replace 'geeks' with 'GeeksforGeeks'.

Common String Methods

Python provides a various built-in methods to manipulate strings. Below are some of the most useful methods.

len(): The len() function returns the total number of characters in a string.

```
s = "GeeksforGeeks"
print(len(s))

# output: 13
```

Output

```
13
```

upper() and lower(): upper() method converts all characters to uppercase. lower() method converts all characters to lowercase.

```
s = "Hello World"

print(s.upper())   # output: HELLO WORLD

print(s.lower())   # output: hello world
```

Output

```
HELLO WORLD

hello world
```

strip() and replace(): strip() removes leading and trailing whitespace from the string and replace(old, new) replaces all occurrences of a specified substring with another.

```
s = "   Gfg   "

# Removes spaces from both ends
print(s.strip())

s = "Python is fun"

# Replaces 'fun' with 'awesome'
print(s.replace("fun", "awesome"))
```

Output

```
Gfg

Python is awesome
```

To learn more about string methods, please refer to Python String Methods.

Concatenating and Repeating Strings

We can concatenate strings using + operator and repeat them using * operator.

Strings can be combined by using **+ operator**.

```
s1 = "Hello"
s2 = "World"
s3 = s1 + " " + s2
print(s3)
```

Output

```
Hello World
```

We can repeat a string multiple times using *** operator**.

```
s = "Hello "
print(s * 3)
```

Output

```
Hello Hello Hello
```

Formatting Strings

Python provides several ways to include variables inside strings.

Using f-strings

The simplest and most preferred way to format strings is by using f-strings.

```
name = "Alice"
age = 22
print(f"Name: {name}, Age: {age}")
```

Output

```
Name: Alice, Age: 22
```

Using format()

Another way to format strings is by using format() method.

```
s = "My name is {} and I am {} years old.".format("Alice", 22)
```

```
print(s)
```

Output

```
My name is Alice and I am 22 years old.
```

Python Classes and Objects

A class in Python is a user-defined template for creating objects. It bundles data and functions together, making it easier to manage and use them. When we create a new class, we define a new type of object. We can then create multiple instances of this object type. Classes are created using **class keyword**. Attributes are variables defined inside the class and represent the properties of the class. Attributes can be accessed using the dot **.** **operator** (e.g., MyClass.my_attribute).

Create a Class

```
# define a class
class Dog:
    sound = "bark"  # class attribute
```

Create Object

An Object is an instance of a Class. It represents a specific implementation of the class and holds its own data.
Now, let's create an object from **Dog class.**

```
class Dog:
    sound = "bark"

# Create an object from the class
dog1 = Dog()

# Access the class attribute
print(dog1.sound)
```

sound attribute is a class attribute. It is shared across all instances of Dog class, so can be directly accessed through instance **dog1**.

Using __init__() Function

In Python, class has __init__() function. It automatically initializes object attributes when an object is created.

```
class Dog:
    species = "Canine"  # Class attribute

    def __init__(self, name, age):
        self.name = name  # Instance attribute
        self.age = age  # Instance attribute
```

Explanation:

- **class Dog:** Defines a class named Dog.
- **species:** A class attribute shared by all instances of the class.
- **__init__ method:** Initializes the name and age attributes when a new object is created.

Initiate Object with __init__

```
class Dog:
    species = "Canine"  # Class attribute

    def __init__(self, name, age):
        self.name = name  # Instance attribute
        self.age = age  # Instance attribute

# Creating an object of the Dog class
dog1 = Dog("Buddy", 3)

print(dog1.name)  # Output: Buddy
print(dog1.species)  # Output: Canine
```

Output

```
Buddy

Canine
```

Explanation:

- **dog1 = Dog("Buddy", 3):** Creates an object of the Dog class with name as "Buddy" and age as 3.
- **dog1.name:** Accesses the instance attribute name of the dog1 object.
- **dog1.species:** Accesses the class attribute species of the dog1 object.

Self Parameter

self parameter is a reference to the current instance of the class. It allows us to access the attributes and methods of the object.

```
class Dog:
    def bark(self):
        print(self.name)

dog1 = Dog("Buddy", 3)
dog1.bark()
```

Explanation:

- **self.name:** Refers to the name attribute of the object (dog1) calling the method.
- **dog1.bark():** Calls the bark method on dog1.

Python Function

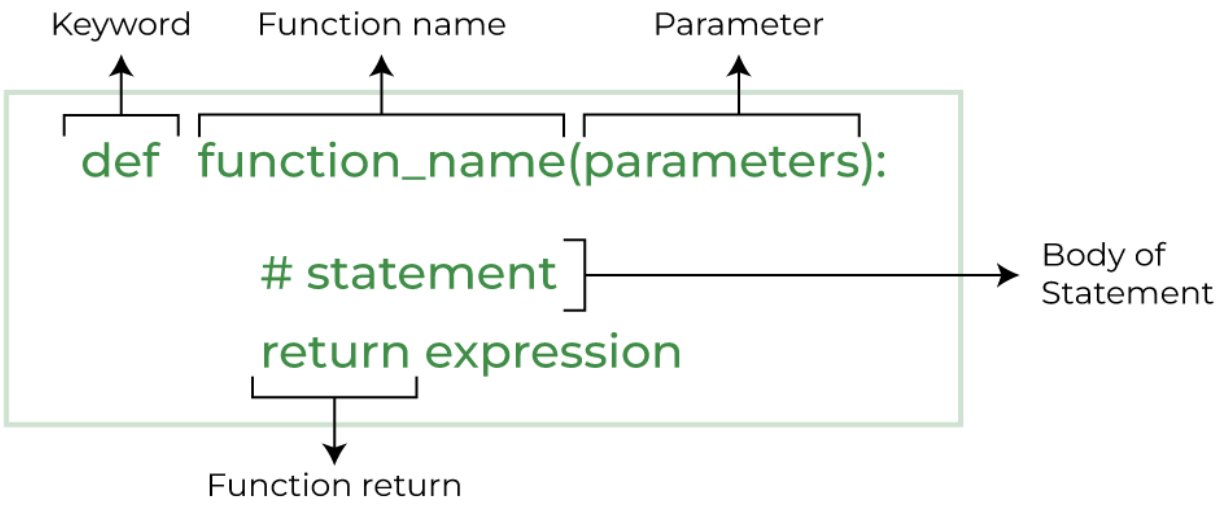
Python Functions is a block of statements that return the specific task. The idea is to put some commonly or repeatedly done tasks together and make a function so that instead of writing the same code again and again for different inputs, we can do the function calls to reuse code contained in it over and over again.

Some **Benefits of Using Functions**

- Increase Code Readability
- Increase Code Reusability

Python Function Declaration

The syntax to declare a function is:



Syntax of Python Function Declaration

Types of Functions in Python

Below are the different types of functions in Python:

- **Built-in library function:** These are Standard functions in Python that are available to use.
- **User-defined function:** We can create our own functions based on our requirements.

Creating a Function in Python

We can define a function in Python, using the **def** keyword. We can add any type of functionalities and properties to it as we require. By the following example, we can understand how to write a function in Python. In this way we can create Python function definition by using def keyword.

```
# A simple Python function
def fun():
    print("Welcome to GFG")
```

Calling a Function in Python

After creating a function in Python we can call it by using the name of the functions Python followed by parenthesis containing parameters of that particular function. Below is the example for calling def function Python.

```
# A simple Python function
def fun():
    print("Welcome to GFG")


# Driver code to call a function
fun()
```

Output:

```
Welcome to GFG
```

Python Function with Parameters

If you have experience in C/C++ or Java then you must be thinking about the *return type* of the function and *data type* of arguments. That is possible in Python as well (specifically for Python 3.5 and above).

Python Function Syntax with Parameters

```
def function_name(parameter: data_type) -> return_type:
    """Docstring"""
    # body of the function
    return expression
```

```
def add(num1: int, num2: int) -> int:
    """Add two numbers"""
    num3 = num1 + num2

    return num3

# Driver code
num1, num2 = 5, 15
ans = add(num1, num2)
print(f"The addition of {num1} and {num2} results {ans}.")
```

Output:

```
The addition of 5 and 15 results 20.
```

Note: *The following examples are defined using syntax 1, try to convert them in syntax 2 for practice.*

```
# some more functions
def is_prime(n):
    if n in [2, 3]:
        return True
    if (n == 1) or (n % 2 == 0):
        return False
    r = 3
    while r * r <= n:
        if n % r == 0:
            return False
        r += 2
    return True
print(is_prime(78), is_prime(79))
```

Output:

```
False True
```

Python Function Arguments

Arguments are the values passed inside the parenthesis of the function. A function can have any number of arguments separated by a comma.

In this example, we will create a simple function in Python to check whether the number passed as an argument to the function is even or odd.

```
# A simple Python function to check
# whether x is even or odd
def evenOdd(x):
```

```
    if (x % 2 == 0):
        print("even")
    else:
        print("odd")


# Driver code to call the function
evenOdd(2)
evenOdd(3)
```

Output:

```
even
odd
```

Types of Python Function Arguments

Python supports various types of arguments that can be passed at the time of the function call. In Python, we have the following function argument types in Python:

- **Default argument**
- **Keyword arguments (named arguments)**
- **Positional arguments**
- **Arbitrary arguments** (variable-length arguments *args and **kwargs)

Let's discuss each type in detail.

Default Arguments

A default argument is a parameter that assumes a default value if a value is not provided in the function call for that argument. The following example illustrates Default arguments to write functions in Python.

```
# Python program to demonstrate
# default arguments
def myFun(x, y=50):
    print("x: ", x)
    print("y: ", y)


# Driver code (We call myFun() with only
# argument)
myFun(10)
```

Output:

```
x:  10
y:  50
```

Like C++ default arguments, any number of arguments in a function can have a default value. But once we have a default argument, all the arguments to its right must also have default values.

Keyword Arguments

The idea is to allow the caller to specify the argument name with values so that the caller does not need to remember the order of parameters.

```
# Python program to demonstrate Keyword Arguments
def student(firstname, lastname):
    print(firstname, lastname)
```

```
# Keyword arguments
student(firstname='Geeks', lastname='Practice')
student(lastname='Practice', firstname='Geeks')
```

Output:

```
Geeks Practice
Geeks Practice
```

Positional Arguments

We used the Position argument during the function call so that the first argument (or value) is assigned to name and the second argument (or value) is assigned to age. By changing the position, or if you forget the order of the positions, the values can be used in the wrong places, as shown in the Case-2 example below, where 27 is assigned to the name and Suraj is assigned to the age.

```
def nameAge(name, age):
    print("Hi, I am", name)
    print("My age is ", age)


# You will get correct output because
# argument is given in order
print("Case-1:")
nameAge("Suraj", 27)
# You will get incorrect output because
# argument is not in order
print("\nCase-2:")
nameAge(27, "Suraj")
```

Output:

```
Case-1:
Hi, I am Suraj
My age is  27
Case-2:
Hi, I am 27
My age is  Suraj
```

Python Inheritance

Inheritance allows us to define a class that inherits all the methods and properties from another class.

Parent class is the class being inherited from, also called base class.

Child class is the class that inherits from another class, also called derived class.

Create a Parent Class

Any class can be a parent class, so the syntax is the same as creating any other class:

Create a class named Person, with firstname and lastname properties, and a printname method:

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname

  def printname(self):
    print(self.firstname, self.lastname)

#Use the Person class to create an object, and then execute the printname method:

x = Person("John", "Doe")
x.printname()
```

Try it Yourself »

Create a Child Class

To create a class that inherits the functionality from another class, send the parent class as a parameter when creating the child class:

Example

Create a class named Student, which will inherit the properties and methods from the Person class:

```
class Student(Person):
  pass
```

Note: Use the pass keyword when you do not want to add any other properties or methods to the class.

Now the Student class has the same properties and methods as the Person class.

Example

Use the Student class to create an object, and then execute the printname method:

```
x = Student("Mike", "Olsen")
x.printname()
```

Add the __init__() Function

So far we have created a child class that inherits the properties and methods from its parent.

We want to add the __init__() function to the child class (instead of the pass keyword).

Note: The __init__() function is called automatically every time the class is being used to create a new object.

Example

Add the __init__() function to the Student class:

```
class Student(Person):
  def __init__(self, fname, lname):
    #add properties etc.
```

When you add the __init__() function, the child class will no longer inherit the parent's __init__() function.

Note: The child's __init__() function **overrides** the inheritance of the parent's __init__() function.

To keep the inheritance of the parent's __init__() function, add a call to the parent's __init__() function:

Example

```
class Student(Person):
  def __init__(self, fname, lname):
    Person.__init__(self, fname, lname)
```

Now we have successfully added the __init__() function, and kept the inheritance of the parent class, and we are ready to add functionality in the __init__() function

Polymorphism in Python

- ---

Polymorphism is a foundational concept in programming that allows entities like functions, methods or operators to behave differently based on the type of data they are handling. Derived from Greek, the term literally means “many forms”.
Python’s dynamic typing and duck typing make it inherently polymorphic. Functions, operators and even built-in objects like loops exhibit polymorphic behavior.
Polymorphism in Built-in Functions
Python’s built-in functions exhibit polymorphism, adapting to various data types.

Example:

```
print(len("Hello"))  # String length
print(len([1, 2, 3]))  # List length

print(max(1, 3, 2))  # Maximum of integers
print(max("a", "z", "m"))  # Maximum in strings
```

Python determines behavior at runtime, enabling these functions to work across diverse types without explicit type declarations.
Let's explore polymorphism in detail:

Polymorphism in Functions

Duck typing enables functions to work with any object regardless of its type.

Example:

```
def add(a, b):
    return a + b

print(add(3, 4))          # Integer addition
print(add("Hello, ", "World!"))  # String concatenation
print(add([1, 2], [3, 4])) # List concatenation
```

Polymorphism in Operators

Operator Overloading

In Python, operators like + behave polymorphically, performing addition, concatenation or merging based on the data type.

Example:

```
print(5 + 10)  # Integer addition
print("Hello " + "World!")  # String concatenation
print([1, 2] + [3, 4])  # List concatenation
```

Polymorphism in OOPs

In OOP, polymorphism allows methods in different classes to share the same name but perform distinct tasks. This is achieved through inheritance and interface design. Polymorphism complements other OOP principles like inheritance (sharing behavior) and encapsulation (hiding complexity) to create robust and modular applications.

Example:

```
class Shape:
    def area(self):
        return "Undefined"

class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width
```

```python
class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius ** 2

shapes = [Rectangle(2, 3), Circle(5)]
for shape in shapes:
    print(f"Area: {shape.area()}")
```

Explanation:

- The code showcases polymorphism using a parent class Shape and child classes Rectangle and Circle.
- **Parent Class Shape:** Contains a generic area method returning "Undefined", acting as a placeholder for derived classes to override.
- **Child Class Rectangle:** Initializes length and width via the __init__ constructor. Overrides the area method to return the rectangle's area as length * width.
- **Child Class Circle:** Initializes radius via the __init__ constructor. Overrides the area method to return the circle's area as 3.14 * radius^2.
- **Polymorphic Behavior:** A list of shape objects (Rectangle and Circle) is created. A for loop iterates through the list, calling the area method on each object. The method executed is determined by the object's type, showcasing polymorphism.

Python Module

Python Module is a file that contains built-in functions, classes,**its** and variables. There are many **Python modules**, each with its specific work.
In this article, we will cover all about Python modules, such as How to create our own simple module, Import Python modules, From statements in Python, we can use the alias to rename the module, etc.

What is Python Module

A Python module is a file containing Python definitions and statements. A module can define functions, classes, and variables. A module can also include runnable code.
Grouping related code into a module makes the code easier to understand and use. It also makes the code logically organized.

Python DateTime module

Python Datetime module supplies classes to work with date and time. These classes provide several functions to deal with dates, times, and time intervals. Date and DateTime are an object in Python, so when you manipulate them, you are manipulating objects and not strings or timestamps.

The DateTime module is categorized into 6 main classes –

- **date** – An idealized naive date, assuming the current Gregorian calendar always was, and always will be, in effect. Its attributes are year, month, and day. you can refer to – Python DateTime – Date Class
- **time** – An idealized time, independent of any particular day, assuming that every day has exactly 24*60*60 seconds. Its attributes are hour, minute, second, microsecond, and tzinfo. You can refer to – Python DateTime – Time Class
- **date-time** – It is a combination of date and time along with the attributes year, month, day, hour, minute, second, microsecond, and tzinfo. You can refer to – Python DateTime – DateTime Class
- **timedelta** – A duration expressing the difference between two date, time, or datetime instances to microsecond resolution. You can refer to – Python DateTime – Timedelta Class
- **tzinfo** – It provides time zone information objects. You can refer to – Python – datetime.tzinfo()
- **timezone** – A class that implements the tzinfo abstract base class as a fixed offset from the UTC (New in version 3.2). You can refer to – Handling timezone in Python

Python Date Class

The date class is used to instantiate date objects in Python. When an object of this class is instantiated, it represents a date in the format **YYYY-MM-DD**. The constructor of this class needs three mandatory arguments year, month, and date.

Python Date class Syntax

```
class datetime.date(year, month, day)
```

The arguments must be in the following range –

- MINYEAR <= year <= MAXYEAR
- 1 <= month <= 12
- 1 <= day <= number of days in the given month and year

Note – If the argument is not an integer it will raise a TypeError and if it is outside the range a ValueError will be raised.

Eg:

```
from datetime import date

my_date = date(1996, 12, 11)

print("Date passed as argument is", my_date)
```

Output:

```
Date passed as argument is 1996-12-11
```

Get the Current Date

To return the current local date today() function of the date class is used. today() function comes with several attributes (year, month, and day). These can be printed individually.

```
from datetime import date

today = date.today()

print("Today's date is", today)
```

Output

```
Today's date is 2025-03-03
```

Get Today's Year, Month, and Date

We can get the year, month, and date attributes from the date object using the year, month and date attribute of the date class.

```
from datetime import date

today = date.today()

print("Current year:", today.year)

print("Current month:", today.month)

print("Current day:", today.day)
```

Output

```
Current year: 2021

Current month: 8

Current day: 19
```

What is a Math Module in Python?

Math Module is an in-built Python library made to simplify mathematical tasks in Python. It consists of various mathematical constants and functions that can be used after importing the math module.

Example: Importing Math Module

```
import math
```

After importing the math module, you can use various constants and functions, let's study some of them

Constants in Math Module

The Python math module provides various values of various constants like **pi**, and **tau**. We can easily write their values with these constants. The constants provided by the math module are :

- Euler's Number
- Pi
- Tau
- Infinity
- Not a Number (NaN)

Let's see constants in math module with examples:

1. Euler's Number

The **math.e** constant returns the Euler's number: 2.71828182846.

Syntax:

math.e

Example: This code imports the math module and then prints the value of the mathematical constant e.

```
import math

print (math.e)
```

Output:

```
2.718281828459045
```

2. Pi

You all must be familiar with pi. The pi is depicted as either 22/7 or 3.14. **math.pi** provides a more precise value for the pi.

Syntax of Python math PI

math.pi

Example 1: This code imports the math module and then prints the value of the mathematical constant **pi**.

```
import math

print (math.pi)
```

Output:

```
3.141592653589793
```

Example 2: Let's find the area of the circle

The code utilizes the math module in Python, defines a radius and the mathematical constant pi, and calculates the area of a circle using the formula' **A = pi * r * r'**. It demonstrates the application of mathematical concepts and the usage of the math module for numerical calculation

```
import math

r = 4

pie = math.pi

print(pie * r * r)
```

Output:

```
50.26548245743669
```

3. tau

Tau is defined as the ratio of the circumference to the radius of a circle. The **math.tau** constant returns the value tau: 6.283185307179586.

Syntax:

math.tau

Example: This code imports the math module and then prints the value of the mathematical constant **'tau'**

```
import math

print (math.tau)
```

Output:

```
6.283185307179586
```

4. Infinity

Infinity basically means something which is never-ending or boundless from both directions i.e. negative and positive. It cannot be depicted by a number. The Python **math.inf** constant returns of positive infinity. For negative infinity, use **-math.inf**.

Syntax:

math.inf

Example 1: This code imports the math module and then prints the values of positive and negative infinity.

```
import math

print (math.inf)

print (-math.inf)
```

Output:

```
inf
-inf
```

Example 2: Comparing the values of infinity with the maximum floating point value

This code imports the math module and then compares the values of positive and negative infinity to the values of 10e108 and -10e108, respectively.

```
import math

print (math.inf > 10e108)

print (-math.inf < -10e108)
```

Output:

```
True
True
```

5. NaN Values

The Python **math.nan** constant returns a floating-point nan (Not a Number) value. This value is not a legal number. The nan constant is equivalent to float("nan").

Example: This code imports the math module and then prints the value of math.nan. math.nan represents Not a Number, which is a special value that is used to indicate that a mathematical operation is undefined or the result is not a number.

```
import math

print (math.nan)
```

Output:

```
nan
```

Python Packages

Python packages are a way to organize and structure code by grouping related modules into directories. A package is essentially a folder that contains an __init__.py file and one or more Python files (modules). This organization helps manage and reuse code effectively, especially in larger projects. It also allows functionality to be easily shared and distributed across different applications. Packages act like toolboxes, storing and organizing tools (functions and classes) for efficient access and reuse.

Key Components of a Python Package

- **Module:** A single Python file containing reusable code (e.g., math.py).
- **Package:** A directory containing modules and a special __init__.py file.
- **Sub-Packages:** Packages nested within other packages for deeper organization.

How to create and access packages in python

1. **Create a Directory:** Make a directory for your package. This will serve as the root folder.
2. **Add Modules:** Add Python files (modules) to the directory, each representing specific functionality.
3. **Include __init__.py:** Add an __init__.py file (can be empty) to the directory to mark it as a package.
4. **Add Sub packages (Optional):** Create subdirectories with their own __init__.py files for sub packages.
5. **Import Modules:** Use dot notation to import, e.g., from mypackage.module1 import greet.
6.

Example :

In this example, we are creating a Math Operation Package to organize Python code into a structured package with two sub-packages: basic (for addition and subtraction) and advanced (for multiplication and division). Each operation is implemented in separate modules, allowing for modular, reusable and maintainable code.

math_operations/__init__.py:

This __init__.py file initializes the main package by importing and exposing the calculate function and operations (add, subtract, multiply, divide) from the respective sub-packages for easier access.

```
# Initialize the main package
from .calculate import calculate
from .basic import add, subtract
from .advanced import multiply, divide
```

math_operations/calculator.py:

This calculate file is a simple placeholder that prints “Performing calculation…”, serving as a basic demonstration or utility within the package.

```
def calculate():
    print("Performing calculation...")
```

math_operations/basic/__init__.py:

This __init__.py file initializes the basic sub-package by importing and exposing the add and subtract functions from their respective modules (add.py and sub.py). This makes these functions accessible when the basic sub-package is imported.

```
# Export functions from the basic sub-package
from .add import add
from .sub import subtract
```

math_operations/basic/add.py:

```
def add(a, b):
    return a + b
```

math_operations/basic/sub.py:

```
def subtract(a, b):
    return a - b
```

In the same way we can create the sub package advanced with multiply and divide modules. Now, let's take an example of importing the module into a code and using the function:

```
from math_operations import calculate, add, subtract

# Using the placeholder calculate function
calculate()

# Perform basic operations
print("Addition:", add(5, 3))
print("Subtraction:", subtract(10, 4))
```

Output:

```
6
8
```